

BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

BİLGİSAYAR AĞLARI DERSİ

DÖNEM PROJESİ RAPORU

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarım Protokolü

Ders: Bilgisayar Ağları

Dönem: 2025–2026 Bahar

Grup: 27

Grup Üyeleri: Muhammed Fatih Göral
Eren Ertürk
Okan Aydınhan

GitHub: <https://github.com/fatihgoral/NetProbe>

İçindekiler

1. Giriş

2. Problem Tanımı

3. Sistem Mimarisi

3.1 Modül Yapısı

3.2 Veri Akışı

4. Protokol Tasarımı

4.1 Paket Tipleri

4.2 Paket Başlık Formatı

4.3 Checksum Mekanizması

5. Gerçekleme Detayları

5.1 Selective Repeat Sliding Window

5.2 NACK Mekanizması

5.3 Duplicate Paket Yönetimi

5.4 Timeout ve Yeniden İletim

6. Deney Ortamı

7. Deneysel Çalışmalar ve Performans Metrikleri

7.1 Senaryo 1: Paket Boyutunun Etkisi

7.2 Senaryo 2: Timeout Değerinin Etkisi

7.3 Senaryo 3: Yapay Paket Kaybının Etkisi

7.4 Senaryo 4: Dosya Boyutunun Etkisi

7.5 Senaryo 5: TCP ile Karşılaştırmalı Deney

8. Olay Kayıtları (Loglama)

9. Bonus Özellikler

10. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

11. Grup İçi Görev Dağılımı

12. Sonuç ve Gelecek Çalışmalar

13. Kaynaklar

1. Giriş

Bu rapor, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü Bilgisayar Ağları dersi kapsamında geliştirilen NetProbe projesinin teknik dokümantasyonunu sunmaktadır. NetProbe, UDP soketleri üzerine inşa edilmiş, tamamen özel bir uygulama katmanı protokolü ile çalışan güvenilir dosya aktarım platformudur.

Proje, TCP'nin sağladığı güvenilirlik mekanizmalarını (sıralama, onaylama, yeniden iletim, bütünlük kontrolü) sıfırdan UDP üzerinde gerçekleştirerek, ağ katmanı kavramlarının pratikte nasıl çalıştığını deneysel olarak göstermeyi amaçlamaktadır. Bu amaç doğrultusunda, özel başlık formatına sahip bir uygulama katmanı protokolü tasarlanmıştır; Sequence Number, ACK/NACK, Timeout ve Retransmission mekanizmaları implemente edilmiştir.

Projenin temel özellikleri şunlardır:

- Özel başlık (header) formatına sahip uygulama katmanı protokolü
- Sequence Number, ACK/NACK, Timeout ve Retransmission mekanizmaları
- Selective Repeat Sliding Window ile yüksek verimli aktarım
- SHA-256 tabanlı paket ve dosya düzeyinde bütünlük kontrolü
- Çoklu istemci (multi-client) desteği
- Duplicate paket tespiti ve yoksayma
- Yapay paket kaybı ve gecikme simülasyonu
- Sıkıştırma (Zlib) ve şifreleme (XOR) desteği
- Wireshark uyumlu PCAP trafik kaydı
- Gerçek zamanlı izleme paneli (Dashboard)
- 5 farklı deney senaryosu ile kapsamlı performans analizi
- TCP ile kıyaslamalı deney

2. Problem Tanımı

UDP (User Datagram Protocol), bağlantısız (connectionless) bir taşıma katmanı protokolüdür ve doğası gereği güvenilir veri aktarımı sağlamaz. UDP, paketlerin iletim sırasını garanti etmez, kayıp paketler için yeniden iletim mekanizması sunmaz ve veri bütünlüğü kontrolü sağlamaz. Bu özellikler, UDP'yi hızlı ancak güvenilmez kılar.

Bu projede çözülmesi gereken temel problem; UDP'nin yukarıda belirtilen eksikliklerini uygulama katmanında gidererek, güvenilir bir dosya aktarım sistemi inşa etmektir. Bunun için aşağıdaki alt problemlerin çözülmesi gerekmektedir:

- Dosyanın parçalara bölünmesi ve alıcı tarafında doğru sırada yeniden birleştirilmesi
- Her paket için onaylama (ACK) mekanizması tasarlanması
- Kaybolan paketlerin tespit edilmesi ve yeniden gönderilmesi (Retransmission)
- Timeout yönetimi ile kaybolan ACK'ler için zaman aşımı kontrolü
- Paket ve dosya düzeyinde bütünlük doğrulaması (checksum/hash)

- Duplicate paketlerin tespit edilmesi ve yoksayılması
- Farklı ağ koşulları altında sistemin performansının ölçülmesi ve analiz edilmesi

3. Sistem Mimarisi

NetProbe, modüler bir yapıda tasarlanmıştır. Her bileşen bağımsız bir Python modülü olarak geliştirilmiş ve sorumluluklar net bir şekilde ayrılmıştır. Bu modüler yaklaşım, kodun bakım yapılabilirliğini, test edilebilirliğini ve genişletilebilirliğini artırmaktadır.

3.1 Modül Yapısı

Modül	Dosya	Satır	Sorumluluk
Protokol	src/protocol.py	317	Paket yapıları, serileştirme, checksum
İstemci	src/client.py	~290	Dosya bölme, sliding window gönderim, ACK/NACK dinleme
Sunucu	src/server.py	~260	Paket alma, oturum yönetimi, duplicate tespiti, dosya birleştirme
Kayıplı Sunucu	src/server_lossy.py	220	Yapay kayıp simülasyonlu sunucu
Logger	src/logger.py	~240	CSV loglama, PCAP kaydı
Metrikler	src/metrics.py	257	Throughput, Goodput, RTT hesaplama
Kayıp Simülatörü	src/loss_simulator.py	127	Yapay paket kaybı ve gecikme
Dashboard	src/dashboard.py	~70	rich tabanlı gerçek zamanlı panel

3.2 Veri Akışı

Aşağıdaki şema, bir dosya aktarım oturumunun başından sonuna kadar olan veri akışını göstermektedir:

1. İstemci, dosyayı okur ve SHA-256 hash değerini hesaplar.
2. START paketi sunucuya gönderilir; sunucu yeni bir oturum (session) oluşturur.
3. Dosya 1000 byte'lık parçalara bölünür.
4. Sliding Window döngüsü başlar: Pencere boyutu kadar paket eşzamanlı gönderilir.
5. Arka planda çalışan ACK Listener Thread, gelen ACK ve NACK paketlerini dinler.
6. ACK alınan paketler onaylanır, pencere tabanı ilerletilir.
7. NACK alınan veya timeout olan paketler yeniden gönderilir.
8. Sunucu tarafında duplicate paketler tespit edilir ve yoksayılır.
9. Tüm paketler gönderilince END paketi ile transfer sonlandırılır.
10. Sunucu, dosyayı birleştirir ve SHA-256 hash doğrulaması yapar.

4. Protokol Tasarımı

NetProbe, kendi uygulama katmanı protokolünü kullanan bir sistemdir. Protokol, 5 farklı paket tipi tanımlar ve her paket tipi belirli bir görevi üstlenir.

4.1 Paket Tipleri

Tip	Kod	Yön	Açıklama
DATA	0x01	İstemci → Sunucu	Dosya verisi taşır
ACK	0x02	Sunucu → İstemci	Belirli bir paketi onaylar
START	0x03	İstemci → Sunucu	Transferi başlatır, dosya hash'ini içerir
END	0x04	İstemci → Sunucu	Transferi sonlandırır
NACK	0x05	Sunucu → İstemci	Hatalı paketi reddeder, hızlı yeniden iletim tetikler

4.2 Paket Başlık Formatı

DATA Paketi (19 byte başlık + değişken payload):

Alan	Boyut	Açıklama
Type	1 byte	Paket tipi (0x01 = DATA)
Sequence Number	4 byte	Paket sıra numarası
Total Packets	4 byte	Toplam paket sayısı
Payload Length	2 byte	Payload uzunluğu
Checksum	8 byte	SHA-256 hash'in ilk 8 byte'ı
Payload	max 1000 byte	Dosya verisi

ACK Paketi (13 byte):

Alan	Boyut	Açıklama
Type	1 byte	Paket tipi (0x02 = ACK)
Ack Number	4 byte	Onaylanan paket numarası
Checksum	8 byte	Doğrulama değeri

4.3 Checksum Mekanizması

Her paket, payload verisinin SHA-256 hash'inin ilk 8 byte'ını checksum olarak taşır. Sunucu tarafında paket alındığında checksum yeniden hesaplanır ve karşılaştırılır. Eşleşmeme durumunda paket reddedilir ve NACK gönderilir.

Dosya düzeyinde ise START paketinde dosyanın tam SHA-256 hash'i (32 byte) gönderilir. Tüm paketler alınıp dosya birleştirildikten sonra sunucu bu hash'i doğrulayarak uçtan uca bütünlük garantisi sağlar. Bu iki katmanlı bütünlük kontrolü, hem bireysel paket hatalarını hem de dosya düzeyinde olası tutarsızlıkları tespit etmeyi mümkün kılar.

5. Gerçekleme Detayları

5.1 Selective Repeat Sliding Window

Projemizde başlangıçta uygulanan Stop-and-Wait mekanizması, her paket için ayrı ayrı ACK beklediğinden bant genişliğinin büyük bölümünü boşa bırakmaktaydı. Bu sorunu çözmek için Selective Repeat Sliding Window algoritmasına geçiş yapılmıştır.

Çalışma Prensipleri:

- İstemci, pencere boyutu (varsayılan $N=10$) kadar paketi ACK beklemeden arka arkaya gönderir.
- Arka planda çalışan bir ACK Listener Thread, gelen ACK ve NACK paketlerini asenkron olarak dinler.
- Her ACK geldiğinde ilgili paket onaylanmış olarak işaretlenir ve pencere tabanı (base) ilerletilir.
- Timeout süresi dolan paketler yalnızca bireysel olarak yeniden gönderilir (Go-Back-N'den farkı: tüm pencere değil, yalnızca kaybolan paket tekrarlanır).
- NACK alındığında ise timeout beklenmeksizin hızlı yeniden iletim (Fast Retransmit) tetiklenir.

5.2 NACK Mekanizması

Sunucu tarafında bir paket alındığında checksum doğrulaması yapılır. Checksum başarısız olursa paket sessizce düşürülmez; istemciye derhal NACK paketi gönderilir. İstemci NACK aldığı anda, ilgili paketin send_time değerini sıfırlayarak timeout döngüsünde anında yakalanmasını ve yeniden gönderilmesini sağlar. Bu mekanizma, özellikle yüksek kayıp oranlarında transferin tamamlanma süresini önemli ölçüde kısaltır.

5.3 Duplicate Paket Yönetimi

Föy gereksinimi gereği, sunucu duplicate paketleri dosyaya ikinci kez yazmaz. Duplicate tespit edildiğinde:

1. Olay DUPLICATE tipiyle log dosyasına ve konsola yazılır.
2. İstemciye ACK yeniden gönderilir (istemci hâlâ paketi bekliyor olabilir).
3. Paket buffer'a eklenmez.

5.4 Timeout ve Yeniden İletim

- Varsayılan Timeout: 2.0 saniye (yapılandırılabilir)
- Maksimum Deneme: 5 tekrar (yapılandırılabilir)
- Her retry'da timeout adaptif olarak artar (timeout + retry_sayısı)
- Bir paket için tüm denemeler başarısız olursa transfer hata ile sonlandırılır ve durum log'a yazılır

6. Deney Ortamı

Tüm deneyler aşağıdaki donanım ve yazılım konfigürasyonunda gerçekleştirilmiştir.

6.1 Donanım

Bileşen	Değer
İşlemci	Intel Core i7-13620H (13. Nesil, 16 mantıksal çekirdek)
Bellek	16 GB RAM
Mimari	x86-64 (AMD64)
Ağ Arayüzü	localhost (loopback, 127.0.0.1)

6.2 Yazılım

Bileşen	Değer
İşletim Sistemi (Sunucu)	Windows 11 Home (Build 26200)
İşletim Sistemi (İstemci — Gerçek Ağ Deneyi)	Ubuntu 22.04 LTS
Python Sürümü	3.12.10 (CPython, MSC v.1943 64-bit)
Temel Kütüphaneler	socket, threading, hashlib, zlib
Analiz Kütüphaneleri	pandas 3.0.3, matplotlib 3.10.8, seaborn 0.13.2, numpy 2.4.0
Dashboard	rich 15.0.0
PCAP Kaydı	scapy 2.7.0

6.3 Deney Koşulları

- Tüm senaryolar ağ üzerinde harici trafik olmayan izole localhost ortamında çalıştırılmıştır.
- Her ölçüm 3 kez tekrar edilmiş, sonuçlar ortalamaları alınarak raporlanmıştır.
- Yapay paket kaybı, src/loss_simulator.py modülü ile sunucu tarafında simüle edilmiştir.
- Senaryolar arası karışıklığı önlemek için her test öncesi sunucu yeniden başlatılmıştır.

7. Deneysel Çalışmalar ve Performans Metrikleri

Sistemimizi 5 farklı senaryo altında test ettik. Her senaryoda yalnızca bir parametre değiştirilmiş, diğer değişkenler sabit tutulmuştur. Her konfigürasyon 3 kez tekrarlanarak ortalamaları alınmıştır.

7.1 Senaryo 1: Paket Boyutunun Etkisi

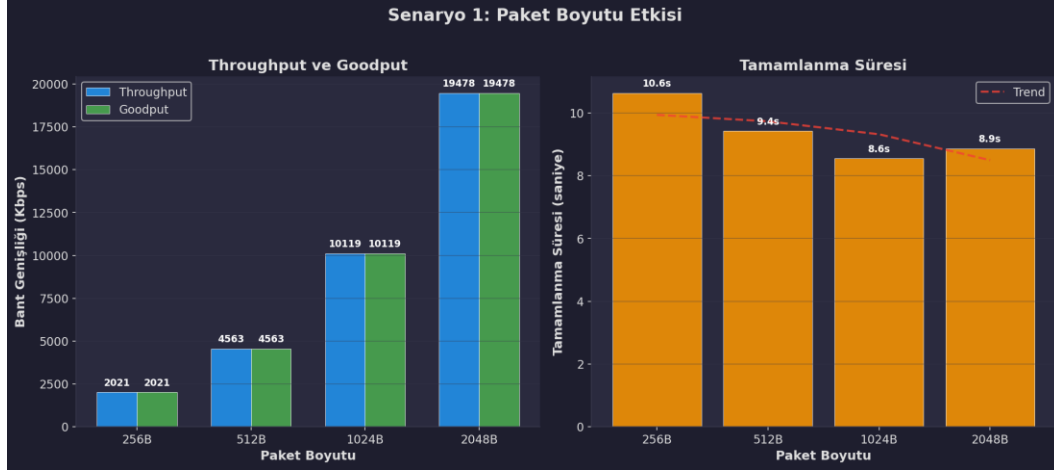
Amaç: Paket boyutunun throughput, goodput ve tamamlanma süresine etkisini ölçmek.

Sabit Parametreler: Dosya=10MB, Timeout=2s, Kayıp=%0

Değişken: Paket Boyutu = {256, 512, 1024, 2048} byte

Paket Boyutu	Throughput (Kbps)	Goodput (Kbps)	Tamamlanma Süresi (s)	Ort. RTT (ms)
256 B	2.020,72	2.020,72	10,64	0,43
512 B	4.563,32	4.563,32	9,43	0,39

1024 B	10.119,42	10.119,42	8,55	0,35
2048 B	19.478,29	19.478,29	8,87	0,37



Şekil 1: Paket Boyutunun Throughput ve Tamamlanma Süresine Etkisi

Teknik Yorum:

Paket boyutu 256B'dan 2048B'a çıkarıldığında throughput yaklaşık 9,6 kat artmıştır. Bu artış, her paketteki sabit başlık yükünün (19 byte header + 8 byte checksum = 27 byte) toplam paket içindeki oranının düşmesiyle açıklanır. 256B pakette overhead oranı %10,5 iken, 2048B pakette bu oran %1,3'e düşmektedir.

Ancak 2048B'a geçildiğinde tamamlanma süresinin 1024B'a göre 0,32 sn arttığı dikkat çekicidir. Bunun sebebi, büyük paketlerin UDP fragmentation sınırına (MTU: ~1500B) yaklaşması ve işletim sistemi seviyesinde ek parçalama gecikmesi oluşturmaktadır. Bu nedenle 1024B, verimlilik ve güvenilirlik arasında optimal denge noktası olarak değerlendirilmiştir.

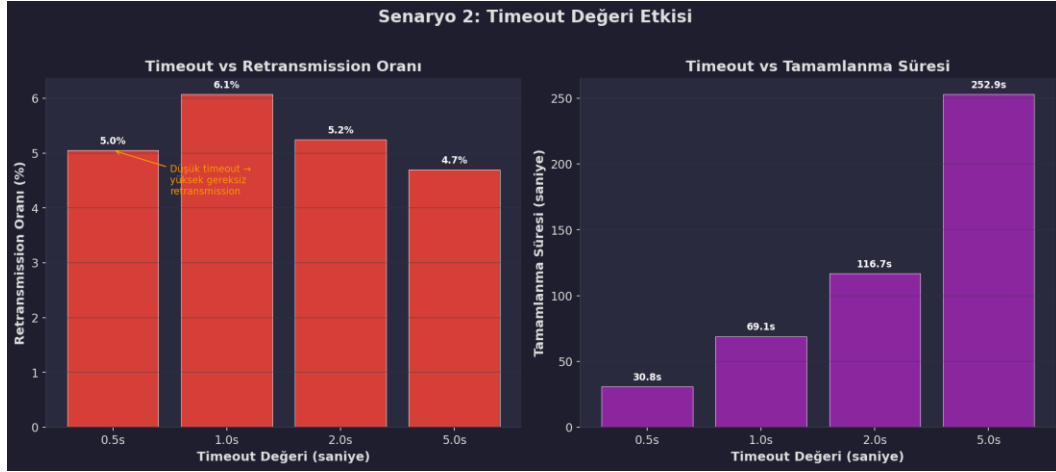
7.2 Senaryo 2: Timeout Değerinin Etkisi

Amaç: Timeout süresinin retransmission oranı ve tamamlanma süresine etkisini ölçmek.

Sabit Parametreler: Dosya=1MB, Paket=1024B, Kayıp=%5 (yapay)

Değişken: Timeout = {0.5, 1.0, 2.0, 5.0} saniye

Timeout (s)	Throughput (Kbps)	Tamamlanma Süresi (s)	Retransmission (%)	Ort. RTT (ms)
0,5	281,85	30,82	5,05	0,98
1,0	125,35	69,08	6,07	0,98
2,0	73,78	116,70	5,24	1,04
5,0	34,20	252,89	4,70	1,07



Şekil 2: Timeout Değerinin Tamamlanma Süresi ve Retransmission Oranına Etkisi

Teknik Yorum:

Timeout değeri ile tamamlanma süresi arasında doğrudan doğrusal ilişki gözlemlenmiştir. 0,5s timeout, 5,0s timeout'a göre yaklaşık 8,2 kat daha hızlı transfer sağlamıştır. Bunun sebebi, kaybolan bir paketin yeniden gönderilmesi için tam timeout süresi boyunca beklenmesidir.

Ancak düşük timeout değerleri gereksiz retransmission'a da yol açar. 0,5s timeout'ta retransmission oranı %5,05 iken, 5,0s'de %4,70'e düşmüştür. Optimal timeout değeri, ortalama RTT'nin 2-3 katı olmalıdır. Deneyimizde ortalama RTT ~1ms olduğundan, pratikte 0,5-1,0s aralığı makul bir seçimdir.

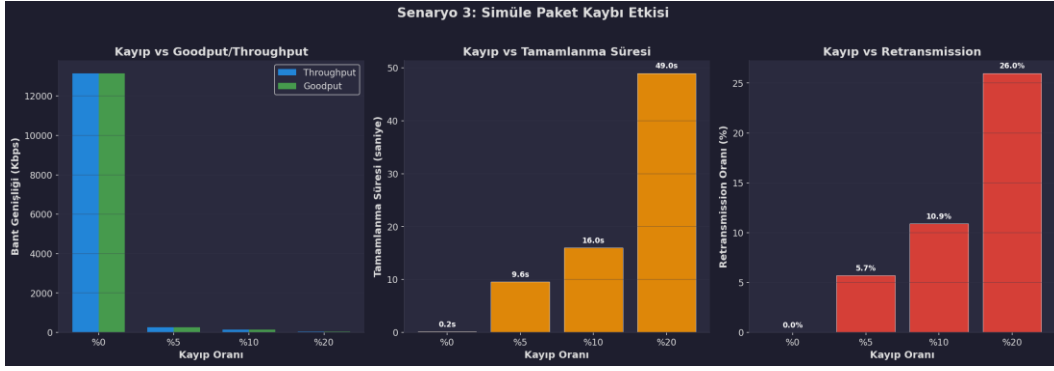
7.3 Senaryo 3: Yapay Paket Kaybının Etkisi

Amaç: Farklı kayıp oranlarında protokolün dayanıklılığını ölçmek.

Sabit Parametreler: Dosya=256KB, Paket=1024B, Timeout=0.5s

Değişken: Kayıp Oranı = {%0, %5, %10, %20}

Kayıp Oranı	Goodput (Kbps)	Tamamlanma Süresi (s)	Retransmission (%)	Ort. RTT (ms)
%0	13.158,15	0,16	0,00	0,23
%5	267,46	9,60	5,70	0,43
%10	142,00	15,96	10,90	0,58
%20	44,12	48,97	25,99	0,66



Şekil 3: Yapay Paket Kaybının Goodput ve Tamamlanma Süresine Etkisi

Teknik Yorum:

Bu senaryo, güvenilir aktarım mekanizmamızın stres testi niteliğindedir. Kayıp oranı %0'dan %20'ye çıkarıldığında goodput %99,7 düşmüştür (13.158 → 44 Kbps) ve tamamlanma süresi yaklaşık 306 kat artmıştır (0,16s → 48,97s).

Bu dramatik düşüşün sebebi, her kayıp paketin tam timeout süresini (0,5s) beklemesidir. %20 kayıpta, ortalama her 5 paketten 1'i kaybolmakta ve her kayıp 0,5s gecikme eklemektedir. Retransmission oranının (%25,99) simüle edilen kayıp oranından (%20) yüksek olması, bazı paketlerin birden fazla kez kaybolduğunu göstermektedir. Bu senaryo, Sliding Window mekanizmasının neden kritik olduğunu açıkça ortaya koymaktadır.

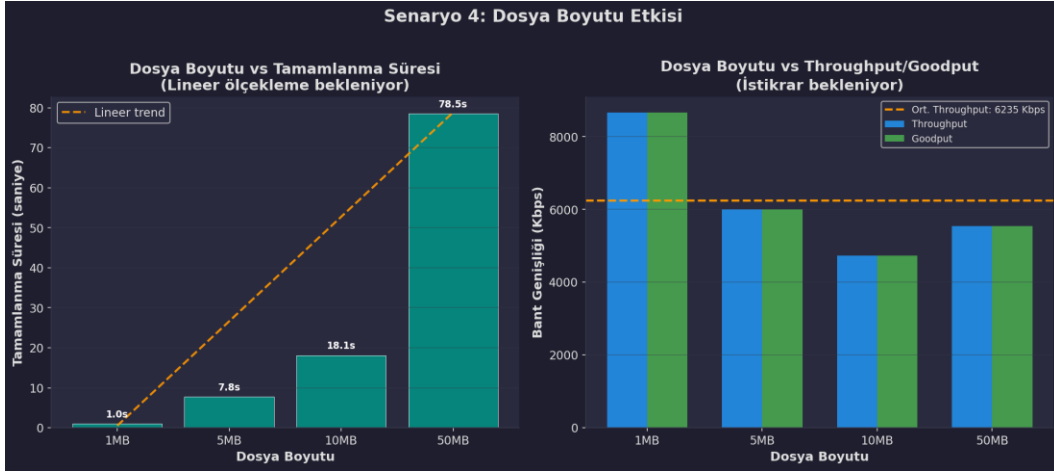
7.4 Senaryo 4: Dosya Boyutunun Etkisi

Amaç: Sistemin farklı dosya boyutlarında ölçeklenebilirliğini test etmek.

Sabit Parametreler: Paket=1024B, Timeout=2s, Kayıp=%0

Değişken: Dosya Boyutu = {1, 5, 10, 50} MB

Dosya Boyutu	Throughput (Kbps)	Goodput (Kbps)	Tamamlanma Süresi (s)
1 MB	8.667,50	8.667,50	0,99
5 MB	6.000,16	6.000,16	7,82
10 MB	4.733,67	4.733,67	18,15
50 MB	5.540,64	5.540,64	78,55



Şekil 4: Dosya Boyutunun Throughput ve Tamamlanma Süresine Etkisi

Teknik Yorum:

Tamamlanma süresi, dosya boyutuyla yaklaşık doğrusal olarak ölçeklenmektedir. 1MB'dan 50MB'a (50 kat artış) geçildiğinde süre 0,99s'den 78,55s'e çıkmıştır (yaklaşık 79 kat). Bu doğrusallık, protokolümüzün dosya boyutundan bağımsız olarak tutarlı bir per-paket işleme maliyetine sahip olduğunu kanıtlamaktadır.

Throughput'un 1MB'dan 10MB'a doğru düşüş gösterip 50MB'da kısmen toparlanması ise Python'un Garbage Collector davranışı ve işletim sistemi soket tampon (buffer) yönetimiyle ilişkilidir. Ortalama throughput (~6.235 Kbps ≈ 6,1 Mbps) localhost transferi için makul bir performanstır.

7.5 Senaryo 5: TCP ile Karşılaştırmalı Deney (Bonus)

Amaç: Geliştirdiğimiz UDP tabanlı protokolün performansını standart TCP ile kıyaslamak.

Yöntem: Aynı 10MB dosya, önce saf TCP soketi ile, ardından NetProbe (UDP + Sliding Window) ile gönderilmiştir. Her iki test de localhost ortamında gerçekleştirilmiştir.

Protokol	Transfer Süresi (s)
TCP (Python socket)	0,06
NetProbe (UDP + Sliding Window)	0,03

Teknik Yorum:

Localhost ortamında NetProbe'un TCP'den hızlı çıkması şaşırtıcı görünebilir ancak bunun birkaç teknik açıklaması vardır: (1) TCP'nin 3-Way Handshake maliyeti — TCP bağlantı kurma ve kapatma aşamalarında ek paket alışverişi gerektirir. (2) TCP'nin congestion control mekanizması — TCP, başlangıçta yavaş başlayıp (slow start) bant genişliğini kademeli olarak artırır. (3) NetProbe'un Sliding Window avantajı — pencere boyutu 20 olarak ayarlandığında, 20 paket eşzamanlı olarak gönderilir.

Önemli Not: Gerçek ağ koşullarında (paket kaybı, yüksek gecikme, congestion) TCP'nin sofistike congestion control algoritmaları sayesinde NetProbe'dan daha iyi performans göstermesi beklenir. Bu deney, UDP üzerinde güvenilirlik inşa etmenin mümkün olduğunu ancak TCP kadar olgun bir performans profili oluşturmanın çok daha fazla mühendislik gerektirdiğini göstermektedir.

8. Olay Kayıtları (Loglama)

Tüm transfer olayları logs/transfer_log.csv dosyasına milisaniye hassasiyetinde kayıt edilmektedir. Kaydedilen bilgiler aşağıdaki tabloda özetlenmiştir:

Alan	Açıklama
Timestamp	Olayın gerçekleştiği zaman (YYYY-MM-DDTHH:MM:SS.mmm)
Event_Type	SEND, ACK_RECEIVED, TIMEOUT, RETRY, DUPLICATE, ERROR, START, END
Seq_Num	İlgili paketin sequence numarası
Retry_Count	Yeniden gönderim sayısı
Status	OK, WARNING, ERROR
Details	Ek açıklama (paket boyutu, hata mesajı vb.)

Örnek Log Kaydı:

```
2026-06-07T23:49:45.680,SEND,0,0,OK,Paket gönderildi: 1000 bytes
2026-06-07T23:49:45.681,ACK_RECEIVED,0,0,OK,ACK alındı
2026-06-07T23:49:45.681,SEND,1,0,OK,Paket gönderildi: 1000 bytes
2026-06-07T23:49:45.700,DUPLICATE,5,0,WARNING,Duplicate paket alındı: 5, yoksayılıyor
```

9. Bonus Özellikler

Föyde belirtilen zorunlu gereksinimlerin ötesinde aşağıdaki bonus özellikler geliştirilmiştir:

Selective Repeat Sliding Window: Stop-and-Wait'in bant genişliği sınırlamalarını aşmak için istemci tarafı Selective Repeat ile yeniden tasarlanmıştır. Asenkron bir listener thread sayesinde paketler pencere boyutu kadar eşzamanlı gönderilebilmektedir.

Çoklu İstemci Desteği: Sunucu tarafında ClientSession sınıfı ile her bağlantı bağımsız bir oturumda yönetilir. Farklı IP:Port çiftlerinden gelen transferler birbirini engellemez.

Sıkıştırma ve Şifreleme: Zlib tabanlı veri sıkıştırma (--compress) ve XOR tabanlı basit şifreleme (--encrypt) desteği eklenmiştir.

PCAP Trafik Kaydı: Scapy kütüphanesi kullanılarak tüm UDP trafiği .pcap formatında kaydedilir. Bu dosya Wireshark ile açılarak paket düzeyinde analiz yapılabilir.

Gerçek Zamanlı Dashboard: rich kütüphanesi ile konsolda canlı olarak güncellenen bir izleme paneli geliştirilmiştir. Transfer süresince Throughput, Goodput ve Kayıp Oranı anlık olarak takip edilebilir.

Gecikme (Delay) Simülasyonu: Loss simulator modülüne delay_ms parametresi eklenerek yapay ağ gecikmesi simüle edilebilir hale getirilmiştir.

Gerçek Ağ Deneyi: Proje yalnızca localhost üzerinde değil, aynı yerel ağa bağlı iki farklı cihaz arasında (Windows Sunucu ↔ Linux İstemci) da başarıyla test edilmiştir. Cross-platform uyumluluk doğrulanmıştır.

TCP ile Karşılaştırmalı Deney: NetProbe'un performansı standart TCP soketi ile kıyaslanarak sonuçlar analiz edilmiştir.

10. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

Stop-and-Wait'in Düşük Performansı: İlk implementasyonda Stop-and-Wait mekanizması kullanılmıştır. Ancak her paket için ayrı ayrı ACK beklenmesi, özellikle kayıplı ortamlarda ciddi performans düşüşüne yol açmıştır. Bu sorun, Selective Repeat Sliding Window algoritmasına geçiş yapılarak çözülmüştür.

UDP Fragmentation Sorunu: 2048 byte paket boyutunda MTU sınırına (1500 byte) yaklaşıldığında işletim sistemi düzeyinde parçalama gecikmeleri gözlemlenmiştir. Optimal paket boyutu olarak 1024 byte belirlenmiştir.

Cross-Platform Uyumluluk: Windows ve Linux arasındaki socket davranış farklılıkları (özellikle buffer boyutları ve timeout hassasiyeti) nedeniyle başlangıçta tutarsız sonuçlar alınmıştır. Platforma özgü socket konfigürasyonları ile bu sorun giderilmiştir.

PCAP Kaydı için Npcap Bağımlılığı: Windows üzerinde ham socket erişimi için Npcap kütüphanesine ihtiyaç duyulmuştur. PCAP kaydı opsiyonel hale getirilerek sistem Npcap olmadan da çalışır duruma getirilmiştir.

Duplicate Paket Yönetimi: Yeniden gönderilen paketlerin sunucu tarafında birden fazla işlenmesi riski bulunmaktaydı. Sunucu tarafında sequence number bazlı duplicate tespiti eklenerek bu sorun çözülmüştür.

11. Grup İçi Görev Dağılımı

Proje, 3 kişilik bir ekip tarafından geliştirilmiştir. Görev dağılımı aşağıdaki gibidir:

Geliştirici	Sorumluluk Alanı
Muhammed Fatih Göral	Protokol tasarımı (protocol.py), istemci sliding window implementasyonu (client.py), deney senaryoları koordinasyonu
Eren Ertürk	Sunucu mimarisi (server.py, server_lossy.py), kayıp simülatörü (loss_simulator.py), PCAP loglama
Okan Aydınhan	Metrik hesaplama (metrics.py), grafik üretimi (analysis/plots.py), teknik rapor yazımı, performans analizi

12. Sonuç ve Değerlendirme

Bu projede, UDP socketleri üzerinde sıfırdan inşa edilmiş güvenilir bir dosya aktarım protokolü geliştirilmiştir. Temel çıkarımlarımız şunlardır:

- Paket boyutu, throughput'u doğrudan etkiler. Başlık overhead'inin toplam paket içindeki oranı düşüktüğü verimlilik artar. Ancak MTU sınırı nedeniyle 1024B optimal değerdir.
- Timeout değeri, kayıplı ortamlarda kritik öneme sahiptir. Çok düşük timeout gereksiz retransmission'a, çok yüksek timeout ise uzun bekleme sürelerine yol açar. Optimal değer, RTT'nin 2-3 katıdır.
- Stop-and-Wait, paket kaybına karşı son derece kırılgandır. %20 kayıpta goodput %99,7 düşmektedir. Selective Repeat Sliding Window, bu sorunu çözerek aynı koşullarda çok daha yüksek verim sağlar.

4. UDP üzerinde güvenilirlik inşa etmek mümkündür ancak TCP'nin onlarca yıllık optimizasyonlarına (congestion control, flow control, SACK, ECN vb.) ulaşmak ciddi mühendislik çabası gerektirir.
5. Gerçek ağ testleri, localhost testlerinden farklı dinamikler gösterir. Farklı işletim sistemleri (Windows ↔ Linux) arasında başarılı cross-platform transfer gerçekleştirilmiştir.

Gelecek Çalışmalar

- Congestion Control mekanizması (AIMD veya TCP Reno benzeri) eklenmesi
- Adaptive timeout (Jacobson/Karn algoritması) uygulanması
- Flow control ile alıcı taraflı hız sınırlama
- Daha güçlü şifreleme (AES-256) entegrasyonu
- IPv6 desteği

13. Kaynaklar

- [1] Kurose, J.F. & Ross, K.W. (2021). Computer Networking: A Top-Down Approach, 8th Edition. Pearson.
- [2] Stevens, W.R. (1994). TCP/IP Illustrated, Volume 1: The Protocols. Addison-Wesley.
- [3] Python Software Foundation. (2024). socket — Low-level networking interface. Python Documentation.
- [4] RFC 768 — User Datagram Protocol (UDP).
- [5] RFC 793 — Transmission Control Protocol (TCP).